

TMC-106

Scripting Languages

M.C.A. (First Semester) - Mid Semester Examination, Nov. 2022

Time: 1½ Hours | Maximum Marks: 50

Complete Solved Question Paper
with detailed answers, programs and revision notes

Instructions in the paper: (i) Answer all the questions by choosing any *one* of the sub-questions. (ii) Each sub-question carries 10 marks. In this document **both** options (a) and (b) of every question are solved so that the full syllabus can be revised.

Q. No.	Topic	Marks	CO
1 (a) / (b)	HTML layout structure, block and inline formatting tags frame vs iframe, frameset program	10	CO1
2 (a) / (b)	meta, list, audio, table, div and span tags image map of India with hot spots	10	CO1
3 (a) / (b)	Ways of applying CSS (inline, internal, external) CSS positioning and z-index	10	CO2
4 (a) / (b)	Pseudo-classes CSS box model with diagram	10	CO2
5 (a) / (b)	Employee login and information forms CSS id and class selectors	10	CO1 / CO2
Bonus	Quick revision notes on the full syllabus	-	-

Note: Image, audio and other files used in the programs (india-map.jpg, tile.png, song.mp3 ...) are placeholders - keep your own files in the same folder as the HTML file. Save each program with the file name given above its code.

Q.1 (a)

Question: Explain the layout structure of HTML document in detail. Classify all the formatting tags in terms of inline and block level elements.

10 Marks | CO1

Answer:

1. Introduction to HTML

HTML (HyperText Markup Language) is the standard markup language used to create web pages. It describes the **structure** of a page with **tags** (elements) written inside angle brackets. A browser reads the HTML file from top to bottom, builds a tree of the elements called the **DOM (Document Object Model)** and displays it. HTML is a *markup* language, not a programming language; it is supported by the scripting languages CSS and JavaScript for presentation and behaviour.

2. Basic layout (skeleton) structure of an HTML document

```
<!DOCTYPE html>                                <!-- 1. document type declaration -->
<html lang="en">                                <!-- 2. root element -->
  <head>                                         <!-- 3. head : information about the page -->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Page</title>
    <link rel="stylesheet" href="style.css">
    <style> p { color: blue; } </style>
    <script src="app.js"></script>
  </head>
  <body>                                       <!-- 4. body : visible content -->
    <h1>Welcome</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

3. Explanation of each part

Part	Description
<!DOCTYPE html>	Tells the browser that the document is HTML5 (so it uses standards mode, not quirks mode). It is not an HTML tag and is not case sensitive.
<html>	Root element that contains the whole page. The <i>lang</i> attribute gives the language (helps screen readers and search engines).
<head>	Container of metadata - information about the page that is not displayed in the browser window (title, character set, styles, scripts).
<body>	Contains everything visible to the user: text, images, links, tables, forms, audio, video, etc.

Elements allowed inside the head

Element	Purpose
<title>	Title shown on the browser tab and in search results (mandatory).
<meta>	Metadata: character set, description, keywords, author, viewport.
<link>	Links external resources - mostly CSS files and favicon.
<style>	Internal CSS written inside the page.
<script>	Internal or external JavaScript.
<base>	Base URL / default target for all relative links of the page.

4. Layout structure of the body using semantic elements

HTML5 provides **semantic elements** whose names describe their role. They divide the page into meaningful areas and are better than a page made only of `<div>` tags for search engines, accessibility and maintenance.

<header> - logo, site title, banner		
<nav> - menu / navigation links		
<aside> - side bar (links, ads)	<main> containing <section> and <article> - the main content of the page	<aside> - related content
<footer> - copyright, contact, links		

Element	Role in the layout
<header>	Introductory area: logo, heading, tagline.
<nav>	Block of navigation links (menu).
<main>	Dominant content of the page (only one per page).
<section>	A thematic group of content, usually with a heading.
<article>	Independent, self-contained content (blog post, news item).
<aside>	Content indirectly related to the main content (sidebar, advertisement).
<footer>	Footer of the page or section - copyright, author, contact info.
<div>	Generic block container used for grouping and styling when no semantic tag fits.

5. Basic rules of HTML syntax

- An element = **start tag + content + end tag**, e.g. `<p>Hello</p>`. Empty (void) elements such as `br`, `hr`, `img`, `input`, `meta`, `link` have no end tag.
- Elements can be **nested** but must be closed in the reverse order of opening.
- **Attributes** give extra information and are written as `name="value"` in the start tag.
- HTML tags are not case sensitive, but lower-case is the convention.
- Comments are written as `<!-- comment -->`.

6. Block-level and inline elements

Block-level elements	Inline elements
Always start on a new line .	Do not start on a new line; flow within the text.
Take the full available width of the parent.	Take only as much width as their content needs.
Width, height, margin and padding (top and bottom) can be set.	Width and height cannot be set; vertical margin has no effect.
Can contain block and inline elements.	Should contain only text/data or other inline elements.
Examples: div, p, h1-h6, ul, ol, li, table, form, pre, hr.	Examples: span, a, b, i, em, strong, img, br, code, sub, sup.
CSS: <i>display: block</i>	CSS: <i>display: inline</i> (a third value, <i>inline-block</i> , mixes both).

7. Classification of formatting tags

Formatting tags change the look or meaning of text. Most of them are **inline**, while the ones that shape paragraphs and blocks of text are **block-level**.

A. Inline formatting tags

Tag	Meaning / effect	Example
	Bold text (visual only).	bold
	Important text (bold, semantic).	strong
<i>	Italic text (visual only).	<i>italic</i>
	Emphasised text (italic, semantic).	emphasis
<u>	Underlined text.	<u>underline</u>
<s> / 	Strike-through / deleted text.	deleted
<ins>	Inserted text (underlined by default).	<u>inserted</u>
<mark>	Highlighted text (yellow background).	highlight
<small>	Smaller text (side comments, fine print).	small
<sub> / <sup>	Subscript / superscript.	H ₂ O, x ²
<code>	Computer code in monospace font.	a = b + c
<kbd> / <samp> / <var>	Keyboard input / sample output / variable.	Ctrl + C
<abbr>	Abbreviation; <i>title</i> gives the full form.	HTML
<cite> / <dfn>	Title of a work / defining term.	<i>Book title</i>
<q>	Short inline quotation (adds quotation marks).	"quote"
<bdo>	Overrides text direction.	reverse text
<time>	Date/time value.	24-Sep-2026
	Generic inline container for styling.	styled text
<a>, , 	Link, image and line break (inline-level).	-
<big>, <tt>, , <strike>	Deprecated in HTML5 - use CSS instead.	-

B. Block-level formatting tags

Tag	Meaning / effect
<h1> ... <h6>	Headings; h1 is the largest and most important, h6 the smallest.
<p>	Paragraph with space before and after.
<pre>	Preformatted text - preserves spaces and line breaks (monospace).
<blockquote>	Long quotation, indented block; <i>cite</i> attribute gives the source.
<address>	Contact information of the author (italic block).
<hr>	Horizontal rule - thematic break between sections.
<div>	Generic block container for grouping content.
<center>	Centres content - deprecated , use CSS text-align.
, , <dl>,	Lists (block-level; li is a list-item block).
<table>, <form>, <figure>	Structured block containers.

8. Example showing formatting tags

```
<!DOCTYPE html>
<html>
<head><title>Formatting tags</title></head>
<body>
  <h1>Formatting Tags</h1>                                <!-- block -->
  <p>This is <b>bold</b>, <i>italic</i>, <u>underlined</u> and <mark>marked</mark>.
    Water is H<sub>2</sub>O and area is a<sup>2</sup>. <del>Old</del> <ins>New</ins>.
    <abbr title="World Wide Web">WWW</abbr> <q>short quote</q>
    <code>x = y + 1;</code>
  </p>
  <pre>
    Preformatted      text
      keeps spaces
  </pre>
  <blockquote cite="http://example.com">A long quotation appears indented.</blockquote>
  <address>Written by MCA Dept., Dehradun</address>
  <hr>
  <div>Block one</div> <div>Block two</div>    <!-- two lines -->
  <span>Inline one</span> <span>Inline two</span>  <!-- same line -->
</body>
</html>
```

Q.1 (b)

Question: Differentiate between <frame> and <iframe> in HTML? Create an html page named as "frame1.html". Divide the page into three columns of 25%, 50% & 75% size. Divide the second 50% size column into two rows of 50%, 50%. In the first 50% size row call the hyperlink "link1.html" and the second 50% size row call the hyperlink "link2.html" and make the pages to be displayed on the column of 25% size and 75% size respectively.

10 Marks | CO1

Answer:

1. Frames in HTML

Frames divide the browser window into several independent rectangular regions; each region loads a separate HTML document. A frame layout is created with <frameset> (which replaces the *body*) and <frame> elements. An <iframe> (inline frame) instead embeds another page **inside the body** of the current page, like an image is embedded. **Note:** <frameset> and <frame> are **obsolete in HTML5** (still working in browsers, but not recommended); <iframe> is the only frame element supported by HTML5.

2. Difference between <frame> and <iframe>

Basis	<frame>	<iframe>
Definition	Defines one window (pane) inside a <frameset>.	Embeds a document inline inside the current page.
Container	Must be placed inside <frameset>, which replaces <body>.	Can be placed anywhere inside <body>.
Layout	Divides the whole window into rows and columns.	Occupies only a rectangle of given width and height within a page.
Closing tag	Empty element - no closing tag.	Has a closing tag </iframe> (fallback text between).
Main attributes	src, name, scrolling, noresize, frameborder, marginwidth, marginheight.	src, name, width, height, srcdoc, sandbox, allow, loading, title.
HTML5 status	Not supported / obsolete.	Supported and widely used.
Nesting	Frames can be nested through nested framesets.	An iframe can contain other iframes.
Body tag	Page with a frameset cannot have a <body>.	Page keeps its normal <body>.
Typical use	Old-style layouts with fixed menu and content pane.	Embedding YouTube videos, Google Maps, ads, widgets, other pages.
Usability / SEO	Poor for bookmarking, printing and search engines.	Better, but content inside is credited to the source page.

3. Attributes of frameset / frame / iframe

Tag	Attribute	Description
<frameset>	cols	Width of each column, in % or px or * (rest), e.g. cols="25%,50%,25%".
<frameset>	rows	Height of each row, e.g. rows="50%,50%".
<frame>	src	Page to display in the frame.
<frame>	name	Name of the frame, used as a <i>target</i> by links.
<frame>	noresize	User cannot drag the frame border.
<frame>	scrolling	yes / no / auto - scroll bar.
<a>	target	Name of the frame (or iframe) in which the linked page opens.

4. Analysis of the question

- The page has **three columns**: 25% (left), 50% (middle), 75% (right). The percentages add up to 150%, so the browser scales them proportionally to 16.7%, 33.3% and 50%. I have used the values exactly as given in the question, and the proportion between columns stays 1 : 2 : 3.
- The **middle column** is split into two rows of 50% each using a nested frameset. Row 1 shows a page with the hyperlink *link1.html*; row 2 shows a page with the hyperlink *link2.html*.
- The frames are named **left** and **right**. The hyperlink in row 1 uses *target="left"* (opens in the 25% column) and the hyperlink in row 2 uses *target="right"* (opens in the 75% column).

5. Program files

File 1: frame1.html (main page)

```
<!DOCTYPE html>
<html>
<head><title>frame1.html</title></head>

<frameset cols="25%,50%,75%">
  <!-- Column 1 : 25% - shows the page opened by link 1 -->
  <frame src="blank1.html" name="left">

  <!-- Column 2 : 50% - divided into two rows of 50% each -->
  <frameset rows="50%,50%">
    <frame src="row1.html" name="menu1">      <!-- contains link 1 -->
    <frame src="row2.html" name="menu2">      <!-- contains link 2 -->
  </frameset>

  <!-- Column 3 : 75% - shows the page opened by link 2 -->
  <frame src="blank2.html" name="right">

  <noframes><body>Your browser does not support frames.</body></noframes>
</frameset>
</html>
```

File 2: row1.html (first 50% row - hyperlink link1.html)

```
<html><body>
  <h3>Row 1</h3>
  <a href="link1.html" target="left">link 1</a>
</body></html>
```

File 3: row2.html (second 50% row - hyperlink link2.html)

```
<html><body>
  <h3>Row 2</h3>
  <a href="link2.html" target="right">link 2</a>
</body></html>
```

Files 4 to 7: link1.html, link2.html, blank1.html, blank2.html

```
<!-- link1.html --> <html><body bgcolor="lightyellow"><h2>This is Link 1 page</h2></body></html>
<!-- link2.html --> <html><body bgcolor="lightgreen"><h2>This is Link 2 page</h2></body></html>
<!-- blank1.html --> <html><body><p>Click "link 1" - page appears here (25%)</p></body></html>
<!-- blank2.html --> <html><body><p>Click "link 2" - page appears here (75%)</p></body></html>
```

6. Working / explanation

- **cols="25%,50%,75%"** creates three vertical columns; the number of frames or nested framesets directly inside must be three.
- The second child of the outer frameset is another **<frameset rows="50%,50%">**, which creates two stacked frames inside the middle column.
- **name** attributes (left, right) identify the frames; the anchor's **target** attribute tells the browser *where* to open the clicked document. Without a target the page would open inside the same frame as the link.
- Result: clicking *link 1* displays link1.html in the 25% column, and clicking *link 2* displays link2.html in the 75% column.

7. Modern (HTML5) alternative using iframes

```
<!DOCTYPE html>
<html>
<head><title>frame1.html</title>
<style>
  .wrap { display: flex; height: 95vh; }
  .wrap iframe { border: 1px solid #555; height: 100%; }
  #left { width: 25%; } #right { width: 75%; }
  .mid { width: 50%; display: flex; flex-direction: column; }
  .mid iframe { height: 50%; }
</style></head>
<body>
<div class="wrap">
  <iframe name="left" id="left"></iframe>
  <div class="mid">
    <iframe src="row1.html"></iframe>      <!-- has link 1, target="left" -->
    <iframe src="row2.html"></iframe>      <!-- has link 2, target="right" -->
  </div>
  <iframe name="right" id="right"></iframe>
</div>
</body>
</html>
```

8. Iframe syntax for revision

```
<iframe src="https://www.example.com" width="600" height="400" name="myFrame"
  title="Example" sandbox loading="lazy">
  Your browser does not support iframes.
</iframe>
<a href="page.html" target="myFrame">Open page in the iframe</a>
```

Q.2 (a)

**Question: Explain the following HTML tags with all the attributes and suitable example: (i) meta information (ii) list (iii) <audio> (iv) <table> (v) <div> and **

10 Marks | CO1

Answer:

(i) Meta information - the <meta> tag

The <meta> tag provides **metadata** (data about the HTML document). It is placed inside the *head* element, is an **empty tag** (no closing tag) and is not displayed on the page. It is used by browsers, search engines and other web services to know the character set, page description, keywords, author, view-port settings and refresh behaviour.

Attribute	Description	Example
charset	Character encoding of the document (UTF-8 is recommended).	<meta charset="UTF-8">
name	Name of the metadata: <i>description, keywords, author, viewport, robots, generator, theme-color</i> .	name="author"
content	Value of the metadata named by <i>name</i> or <i>http-equiv</i> .	content="MCA Dept"
http-equiv	Acts like an HTTP header: <i>refresh, content-type, content-security-policy, x-ua-compatible</i> .	http-equiv="refresh" content="30"

```
<head>
  <meta charset="UTF-8">                                <!-- encoding -->
  <meta name="description" content="Notes on HTML for MCA students"> <!-- SEO -->
  <meta name="keywords" content="HTML, CSS, JavaScript, MCA"> <!-- search words -->
  <meta name="author" content="Rahul Sharma"> <!-- author name -->
  <meta name="robots" content="index, follow"> <!-- crawler rule -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0"> <!-- mobile -->
  <meta http-equiv="refresh" content="30"> <!-- reload page every 30 sec -->
  <meta http-equiv="refresh" content="5; url=https://example.com"> <!-- redirect -->
</head>
```

(ii) Lists

Lists group related items. HTML supports three types: **unordered list** (bullets), **ordered list** (numbers/letters) and **description list** (term and description).

Tag	Purpose	Attributes
	Unordered (bulleted) list.	<i>type</i> (disc, circle, square) - deprecated; use CSS <i>list-style-type</i> .
	Ordered (numbered) list.	<i>type</i> (1, A, a, I, i), <i>start</i> (first number), <i>reversed</i> (count down).
	List item inside ul or ol.	<i>value</i> (number of this item in an ol).
<dl>	Description list.	-
<dt>	Term being described.	-
<dd>	Description of the term.	-

```
<h3>Unordered</h3>
<ul type="square">
  <li>HTML</li>
  <li>CSS
```

```

        <ul><li>Selectors</li><li>Box model</li></ul>      <!-- nested list -->
    </li>
</ul>

<h3>Ordered</h3>
<ol type="A" start="3" reversed>
    <li>First</li> <li>Second</li> <li value="10">Item numbered 10</li>
</ol>

<h3>Description list</h3>
<dl>
    <dt>HTML</dt> <dd>HyperText Markup Language</dd>
    <dt>CSS</dt> <dd>Cascading Style Sheets</dd>
</dl>

```

(iii) The <audio> tag

The <audio> element (HTML5) embeds sound content such as music or podcasts. Fallback text is shown if the browser does not support it. Multiple <source> elements offer different formats (MP3 - *audio/mpeg*, OGG - *audio/ogg*, WAV - *audio/wav*); the browser plays the first supported one.

Attribute	Description
src	URL of the audio file.
controls	Displays play/pause, seek bar and volume controls.
autoplay	Starts playing automatically (most browsers need muted).
loop	Replays the audio again and again.
muted	Starts with the sound off.
preload	none metadata auto - how much of the file is loaded in advance.
crossorigin	CORS setting for a file on another domain.

```

<audio controls loop preload="auto">
    <source src="song.mp3" type="audio/mpeg">
    <source src="song.ogg" type="audio/ogg">
    Your browser does not support the audio tag.
</audio>

```

(iv) The <table> tag

A table arranges data into **rows and columns**. It is built from the following elements:

Tag	Purpose
<table>	Defines the table.
<caption>	Title of the table.
<thead> / <tbody> / <tfoot>	Header, body and footer groups of rows.
<tr>	Table row.
<th>	Header cell - bold and centred by default.
<td>	Data cell.
<colgroup> / <col>	Groups columns to apply styles to whole columns.

Attribute	Used with	Description
border	table	Width of the table border in pixels (use CSS border in HTML5).
cellpadding	table	Space between cell content and its border (deprecated - use padding).

Attribute	Used with	Description
cellspacing	table	Space between cells (deprecated - use border-spacing).
width, height	table, td, th	Size of table or cell in px or %.
align, valign	table, tr, td	Horizontal / vertical alignment (use CSS text-align, vertical-align).
bgcolor	table, tr, td	Background colour (deprecated - use CSS background-color).
colspan	td, th	Number of columns a cell spans.
rowspan	td, th	Number of rows a cell spans.
scope	th	col, row, colgroup, rowgroup - which cells the header belongs to.
headers	td	IDs of header cells the data cell is related to (accessibility).
nowrap	td, th	Stops text wrapping (deprecated).

```
<table border="1" cellpadding="6" cellspacing="0" width="70%">
  <caption>Student Result</caption>
  <thead>
    <tr><th>Roll No</th><th>Name</th><th colspan="2">Marks</th></tr>
  </thead>
  <tbody>
    <tr><td>1</td><td>Amit</td><td>Maths 85</td><td>Physics 78</td></tr>
    <tr><td>2</td><td>Neha</td><td rowspan="2">Absent</td><td>Chem 90</td></tr>
    <tr><td>3</td><td>Ravi</td><td>Bio 70</td></tr>
  </tbody>
  <tfoot><tr><td colspan="4">End of result</td></tr></tfoot>
</table>
```

(v) The <div> and tags

Both are **generic containers** with no meaning of their own. They are used to group elements so that CSS or JavaScript can be applied to the group. They support the global attributes *id*, *class*, *style*, *title*, *lang*, *hidden*, *data-**, *onclick* etc., and *align* (deprecated) on *div*.

Basis	<div>	
Type	Block-level element.	Inline element.
New line	Starts on a new line and takes the full width.	Stays in the same line and takes only the content width.
Use	Grouping large sections - layout, containers, page structure.	Styling a small part of text, a word or a phrase.
Can contain	Block and inline elements.	Only inline elements and text.
Width / height	Can be set.	Cannot be set (unless display:inline-block).
Example	<div id="header">...</div>	text

```
<style>
  .card { border: 2px solid navy; padding: 10px; width: 300px; background: #eef; }
  .red { color: red; font-weight: bold; }
</style>

<div class="card">
  <h3>Notice</h3>
  <p>Exam starts on <span class="red">25 November</span> at 10 AM.</p>
</div>
```

Q.2 (b)

Question: Embed the map of India on the web page and create hot spot on the states UP and Uttarakhand using circular shape area and a rectangular shape area with title UP and Uttarakhand while selecting, and link the states with the official websites of the respective states. Assume dummy coordinates of the states.

10 Marks | CO1

Answer:

1. What is an image map?

An **image map** is a single image in which several regions, called **hot spots**, act as separate hyperlinks. Clicking on a region takes the user to a different URL. The map of India is embedded with the `<img>` tag and connected to a `<map>` element containing one `<area>` tag for every hot spot.

2. Tags and attributes used

Tag / attribute	Description
<code></code>	Embeds the image; <i>usemap</i> connects it to the map having the same name (with # symbol).
<code><map name="name"></code>	Defines the image map; contains area elements. The <i>name</i> must match the usemap value.
<code><area></code>	Defines one clickable region. It is an empty element.
<code>shape</code>	Shape of the hot spot: <i>rect</i> , <i>circle</i> , <i>poly</i> , <i>default</i> .
<code>coords</code>	Coordinates in pixels measured from the top-left corner of the image.
<code>href</code>	Destination URL of the hot spot.
<code>alt</code>	Alternate text (required when href is used).
<code>title</code>	Tool-tip text shown when the mouse pointer is placed on the region.
<code>target</code>	Where to open the link: <i>_blank</i> (new tab), <i>_self</i> , <i>_parent</i> , <i>_top</i> .

3. Shapes and their coordinates

Shape	coords format	Explanation
<code>rect</code>	<code>x1, y1, x2, y2</code>	Top-left corner (x1, y1) and bottom-right corner (x2, y2).
<code>circle</code>	<code>x, y, radius</code>	Centre of the circle (x, y) and its radius.
<code>poly</code>	<code>x1,y1, x2,y2, x3,y3, ...</code>	Corner points of a polygon (irregular shape such as a state).
<code>default</code>	<code>no coords</code>	Entire remaining area of the image.

4. Program

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Map of India - Image Map</title>
</head>
<body>
  <h2>Map of India - click on Uttar Pradesh or Uttarakhand</h2>

  <map name="indiamap">
    <!-- Uttar Pradesh : circular hot spot (dummy centre 290,260 and radius 45) -->
    <area shape="circle" coords="290,260,45"
      href="https://up.gov.in" target="_blank"
      alt="Uttar Pradesh" title="UP">

    <!-- Uttarakhand : rectangular hot spot (dummy corners 310,150 and 380,205) -->
    <area shape="rect" coords="310,150,380,205"
      href="https://uk.gov.in" target="_blank"
      alt="Uttarakhand" title="Uttarakhand">
  </map>
</body>
</html>
```

5. Explanation

- **india-map.jpg** is the map image saved in the same folder (any map of India can be used). *width* and *height* fix the display size, so the coordinates stay correct.
- **usemap="#indiamap"** links the image to **<map name="indiamap">**.
- The first **area** uses *shape="circle"* with *coords="290,260,45"* (centre x = 290, y = 260, radius = 45) for **Uttar Pradesh**; *title="UP"* shows a tool-tip "UP" when the pointer is on the hot spot; *href* links to the UP government portal.
- The second **area** uses *shape="rect"* with *coords="310,150,380,205"* for **Uttarakhand** with *title="Uttarakhand"* and the Uttarakhand government portal.
- **target="_blank"** opens the official website in a new browser tab.
- The coordinates are dummy values. To find the real coordinates, open the image in an editor (e.g. Paint) and note the pixel positions, or use an online image-map generator.

Note: The official sites used are the state portals *up.gov.in* (Uttar Pradesh) and *uk.gov.in* (Uttarakhand). Please check that the links still open before submitting.

6. Extra: polygon hot spot and default area

```
<area shape="poly" coords="300,120, 340,110, 380,140, 350,190, 300,170"
  href="https://uk.gov.in" alt="Uttarakhand (irregular shape)" title="Uttarakhand">
<area shape="default" href="https://www.india.gov.in" alt="India" title="India">
```

7. Advantages and limitations of image maps

Advantages	Limitations
Many links from a single image; fewer HTTP requests.	Coordinates do not scale automatically when the image is resized (not responsive).
Useful for maps, floor plans and diagrams.	Finding coordinates for irregular shapes is time consuming.
Tool-tips through the title attribute.	Needs alt text for every area for accessibility; inline SVG is a modern alternative.

Q.3 (a)

Question: How many ways you can apply CSS styles to a web page. Create a web page which includes following properties on any element: (i) Font color, font size and font family using inline CSS. (ii) Text decoration, text shadow and text transform using internal CSS. (iii) Background color and background repeat using External CSS.

10 Marks | CO2

Answer:

1. What is CSS?

CSS (Cascading Style Sheets) is a style sheet language used to describe the presentation (colours, fonts, spacing, layout) of an HTML document. It separates **content (HTML)** from **design (CSS)**, so the look of a whole website can be changed by editing one file. A CSS rule has a **selector** and a **declaration block** of property : value pairs:

```
selector {  
    property : value ;      /* declaration */  
    color    : red ;  
}
```

2. Ways to apply CSS to a web page

There are **three** main ways to add CSS to an HTML page (a fourth, @import, is a variation of the external method):

(a) Inline CSS

Written in the **style attribute** of a single HTML element. It affects only that element.

```
<p style="color:red; font-size:20px;">This paragraph is styled inline.</p>
```

(b) Internal (embedded) CSS

Written inside a **<style>** element in the *head* section. It applies to the whole page.

```
<head>  
  <style>  
    p { color: blue; }  
    h1 { text-align: center; }  
  </style>  
</head>
```

(c) External CSS

Written in a separate file with the **.css** extension and linked to the page using **<link>** in the head. One file can control many pages.

```
<head>  
  <link rel="stylesheet" type="text/css" href="style.css">  
</head>
```

(d) Imported CSS (@import)

```
<style>  
  @import url("style.css");      /* imports an external file inside a style block */  
</style>
```

3. Comparison of the three methods

Basis	Inline CSS	Internal CSS	External CSS
Where written	In the style attribute of the tag.	In the <style> tag inside <head>.	In a separate .css file linked with <link>.
Scope	Only one element.	One web page.	Any number of pages.
Reusability	None.	Only within the same page.	Highly reusable.
Maintenance	Difficult, mixes content and design.	Moderate.	Easiest - change one file.
Priority (specificity)	Highest.	Medium.	Lowest (if declared first).
Page load	No extra request.	No extra request.	One more HTTP request (but cached by the browser).
Best used for	Quick tests, one-off changes.	Single-page sites.	Full websites.

4. Priority of CSS (cascading order)

- Inline style > internal / external style (the one written *later* wins when both have equal specificity) > browser default style.
- Specificity order: **!important** > inline > ID selector > class / attribute / pseudo-class > element selector.

5. Program using all three methods

File 1: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Ways to apply CSS</title>

  <!-- (iii) EXTERNAL CSS : background color and background repeat -->
  <link rel="stylesheet" href="style.css">

  <!-- (ii) INTERNAL CSS : text decoration, text shadow, text transform -->
  <style>
    .internal {
      text-decoration: underline overline; /* underline + overline */
      text-shadow: 3px 3px 5px gray; /* x-offset y-offset blur color */
      text-transform: uppercase; /* none | capitalize | lowercase */
      font-size: 24px;
    }
  </style>
</head>
<body>

  <!-- (i) INLINE CSS : font color, font size and font family -->
  <h1 style="color: darkgreen; font-size: 36px; font-family: 'Times New Roman', serif;">
    Inline CSS Heading
  </h1>

  <p class="internal">this text is styled with internal css</p>

  <div class="box">This div uses external CSS: background colour and background repeat.</div>

</body>
</html>
```

File 2: style.css (external style sheet)

```
body {  
    background-color: #fff8dc;           /* page background colour */  
}  
.box {  
    height: 200px;  
    border: 2px solid #333;  
    background-color: lightblue;        /* background colour */  
    background-image: url("tile.png");  /* small picture to be tiled */  
    background-repeat: repeat-x;        /* repeat | repeat-x | repeat-y | no-repeat */  
    font-size: 18px;  
}
```

6. Explanation of the properties used

Property	Values	Effect
color	name, hex, rgb()	Colour of the text.
font-size	px, em, rem, %, keywords	Size of the text.
font-family	list of fonts	Typeface; last value is a generic family (serif, sans-serif, monospace).
text-decoration	none, underline, overline, line-through	Lines drawn on the text.
text-shadow	h-offset v-offset blur colour	Shadow behind the text.
text-transform	none, uppercase, lowercase, capitalize	Changes the case of the letters.
background-color	colour	Solid background colour.
background-repeat	repeat, repeat-x, repeat-y, no-repeat, space, round	Whether/how a background image is tiled. Works only when a background-image is present.

Note: tile.png is a small placeholder image kept in the same folder as style.css. Without a background image, background-repeat has no visible effect.

Q.3 (b)

Question: Create a web page which contains four <div> elements and implement all CSS positioning properties to properly position these elements and create layers of elements from one to four where the z-index can vary accordingly.

10 Marks | CO2

Answer:

1. CSS positioning

The CSS **position** property decides how an element is placed in the document. Together with the offset properties **top**, **right**, **bottom**, **left** it controls the exact location of the element. The **z-index** property controls the stacking order of overlapping positioned elements (which one appears on top).

2. Values of the position property

Value	Behaviour
static	Default. Element follows the normal document flow. top, right, bottom, left and z-index have no effect .
relative	Positioned relative to its own normal position . The original space is kept reserved. Also becomes a reference for absolute children.
absolute	Removed from the normal flow and positioned relative to the nearest positioned ancestor (an ancestor whose position is not static); if none, relative to the page (initial containing block).
fixed	Positioned relative to the browser viewport . It stays at the same place even when the page is scrolled.
sticky	Hybrid: behaves as relative until a scroll threshold (e.g. top: 0) is reached and then sticks like fixed inside its parent.

3. Related properties

Property	Description
top, right, bottom, left	Offset of the element from the corresponding edge of its reference box. Work only if position is not static.
z-index	Integer stack level. Higher value = nearer to the viewer. Works only on positioned elements (or flex/grid items). Can be negative.
overflow	visible, hidden, scroll, auto - what happens when content is larger than the box.
clip / clip-path	Shows only a part of an absolutely positioned element.
float / clear	Older layout method that takes an element out of flow to the left or right.

4. Program: four layered <div> elements (z-index 1 to 4)

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>CSS Positioning and z-index</title>
<style>
  body { font-family: Arial, sans-serif; height: 1500px; } /* tall page to scroll */

  h2.sticky { position: sticky; top: 0; background: #333; color: #fff;
             padding: 8px; margin: 0; z-index: 10; } /* sticky heading */

  .container { position: relative; /* reference for absolute children */
              width: 520px; height: 330px; margin: 20px;
              border: 3px dashed #666; }

  .box { width: 200px; height: 130px; color: #fff; font-weight: bold;
        padding: 8px; box-sizing: border-box; border: 2px solid #000; }

  /* DIV 1 : relative - moved 20px from its normal position, layer 1 (bottom) */
  #d1 { position: relative; top: 20px; left: 20px; z-index: 1; background: crimson; }

  /* DIV 2 : absolute - inside .container, layer 2 */
  #d2 { position: absolute; top: 70px; left: 110px; z-index: 2; background: seagreen; }

  /* DIV 3 : absolute - overlaps div 2, layer 3 */
  #d3 { position: absolute; top: 120px; left: 200px; z-index: 3; background: royalblue; }

  /* DIV 4 : fixed - relative to the browser window, layer 4 (top) */
  #d4 { position: fixed; bottom: 20px; right: 20px; z-index: 4; background: darkorange; }
</style>
</head>
<body>
  <h2 class="sticky">position: sticky heading (stays at the top while scrolling)</h2>

  <p>This paragraph is <b>static</b> - normal flow (default position).</p>

  <div class="container">
    <div class="box" id="d1">DIV 1<br>relative, z-index: 1</div>
    <div class="box" id="d2">DIV 2<br>absolute, z-index: 2</div>
    <div class="box" id="d3">DIV 3<br>absolute, z-index: 3</div>
  </div>

  <div class="box" id="d4">DIV 4<br>fixed, z-index: 4</div>

  <p>Scroll down: DIV 4 remains fixed at the bottom-right corner of the window.</p>
</body>
</html>
```

5. Explanation of the program

Element	Positioning	Layer	How it behaves
Paragraph	static (default)	-	Stays in normal flow; offsets and z-index are ignored.
Heading	sticky; top: 0	10	Scrolls with the page until it reaches the top, then sticks.
#d1	relative; top 20px; left 20px	1 (bottom)	Shifted from its normal place, its old space stays reserved. Lowest layer, so it is covered by #d2.
#d2	absolute; top 70px; left 110px	2	Positioned inside .container (which is relative). Covers #d1, is covered by #d3.
#d3	absolute; top 120px; left 200px	3	Overlaps #d2 and #d1 partly; sits above them.
#d4	fixed; bottom 20px; right 20px	4 (top)	Fixed to the window corner; highest layer, so it is always above the other boxes.

- **.container {position: relative}** is essential: without it, #d2 and #d3 would be positioned relative to the whole page instead of the container.
- The **stacking order** is decided by z-index: $4 > 3 > 2 > 1$. If two elements have the same z-index, the one that comes *later* in the HTML appears on top.
- To change layers, only the z-index numbers have to be changed - e.g. z-index: 5 on #d1 brings it to the top.

6. Difference between relative and absolute (viva point)

Basis	relative	absolute
Reference point	Its own original position.	Nearest positioned ancestor (or page).
Normal flow	Stays in the flow; original space is kept.	Removed from the flow; no space is kept.
Width	Behaves as normal block.	Shrinks to fit content unless width is given.
Use	Small shifts; parent of absolute elements.	Overlays, badges, pop-ups, layered designs.

Q.4 (a)

Question: What do you mean by pseudo classes in CSS? Apply the pseudo classes on the following: (i) Apply pseudo classes for links (ii) Apply pseudo classes for 2nd and 4th item of the list

10 Marks | CO2

Answer:

1. Meaning of pseudo-class

A **pseudo-class** is a keyword added to a selector, after a **colon (:)**, that specifies a **special state or condition** of the selected element. It lets us style an element based on user action (hover, focus), its position in the document (first child, 2nd item) or its state (checked, disabled, visited) - things that cannot be selected using ordinary class or id.

```
selector:pseudo-class {  
    property: value;  
}  
a:hover { color: red; }          /* when the mouse is over a link */
```

2. Types of pseudo-classes

Category	Pseudo-classes	Meaning
Link	:link, :visited	Unvisited link / already visited link.
User action	:hover, :active, :focus	Mouse over / being clicked / has keyboard focus.
Structural	:first-child, :last-child, :only-child, :nth-child(n), :nth-last-child(n), :nth-of-type(n), :first-of-type, :last-of-type, :empty, :root	Selected by position in the DOM tree.
Form / input	:checked, :disabled, :enabled, :required, :optional, :valid, :invalid, :read-only, :in-range	State of form controls.
Negation / other	:not(selector), :target, :lang(), :is(), :where()	Exclusion, URL fragment target, language etc.

3. (i) Pseudo-classes for links

Pseudo-class	Applies when
a:link	The link has not been visited yet.
a:visited	The user has already visited the link.
a:hover	The mouse pointer is over the link.
a:active	The link is being clicked (mouse button pressed).
a:focus	The link is selected using the keyboard (Tab key).

Order rule (LoVe HATe): a:link, a:visited, a:hover, a:active. a:hover must come after a:link and a:visited, and a:active must come after a:hover; otherwise the later rules override earlier ones.

4. (ii) Pseudo-classes for the 2nd and 4th list item

The **:nth-child(n)** pseudo-class selects an element that is the n-th child of its parent. It accepts a number, a keyword (*odd*, *even*) or a formula ($an+b$).

Selector	Selects
li:nth-child(2)	The 2nd list item.
li:nth-child(4)	The 4th list item.
li:nth-child(even)	2nd, 4th, 6th... (2n).
li:nth-child(odd)	1st, 3rd, 5th... (2n+1).
li:nth-child(3n)	3rd, 6th, 9th ...
li:nth-child(n+3)	3rd item onward.
li:first-child / li:last-child	First / last item.

5. Program

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Pseudo classes</title>
<style>
/* ----- (i) pseudo-classes for links (keep this order) ----- */
a:link { color: blue; text-decoration: none; } /* unvisited */
a:visited { color: purple; } /* visited */
a:hover { color: red; text-decoration: underline; background: #ffff99; } /* mouse over*/
a:active { color: orange; } /* clicked */
a:focus { outline: 2px solid green; } /* keyboard */

/* ----- (ii) pseudo-classes for 2nd and 4th list item ----- */
ul { width: 250px; }
li:nth-child(2) { background: lightgreen; font-weight: bold; } /* 2nd item */
li:nth-child(4) { background: lightsalmon; color: white; font-style: italic; } /* 4th item */

/* extra examples */
li:first-child { border-top: 3px solid black; }
li:last-child { border-bottom: 3px solid black; }
li:hover { background: yellow; }
</style>
</head>
<body>
<h2>Links</h2>
<a href="https://www.google.com">Google</a> |
<a href="https://www.wikipedia.org">Wikipedia</a> |
<a href="page.html">Local page</a>

<h2>List</h2>
<ul>
<li>Item 1</li>
<li>Item 2 (2nd - green)</li>
<li>Item 3</li>
<li>Item 4 (4th - orange)</li>
<li>Item 5</li>
</ul>
</body>
</html>
```

6. Explanation

- **a:link** styles new links; **a:visited** changes the colour after the page has been visited (browsers allow only limited properties, such as colour, for privacy reasons).

- **a:hover** changes the colour and adds an underline and a yellow background when the mouse moves over the link; **a:active** shows orange at the moment of clicking.
- **li:nth-child(2)** and **li:nth-child(4)** give the second and fourth items different background colours and text styles. The selectors *li:nth-child(2)*, *li:nth-child(4)* can be combined, or *li:nth-child(even)* used for both when the list has only 4 items.

7. Pseudo-class vs pseudo-element

Basis	Pseudo-class	Pseudo-element
Syntax	Single colon, e.g. <i>a:hover</i> .	Double colon, e.g. <i>p::first-line</i> .
Selects	An element in a special state .	A part of an element, or generated content.
Examples	hover, focus, nth-child, checked.	::before, ::after, ::first-letter, ::first-line, ::selection, ::placeholder.

Q.4 (b)

Question: What is the importance of box model in CSS, explain with proper diagram. Develop a web page to demonstrate all the box model properties on the <div> element.

10 Marks | CO2

Answer:

1. What is the CSS box model?

In CSS every HTML element is treated as a **rectangular box**. The **box model** describes how the size and the space of this box are calculated. The box is made of four layers, from the inside to the outside: **Content**, **Padding**, **Border** and **Margin**.

2. Diagram of the box model

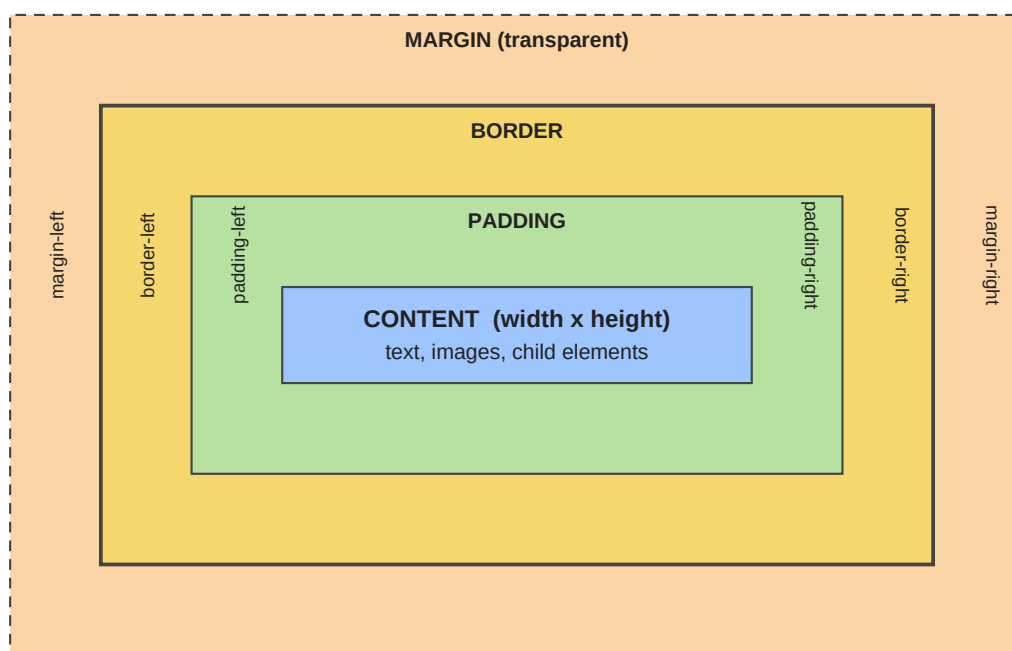


Figure: CSS box model - content is surrounded by padding, border and margin.

3. Parts of the box model

Layer	Description	Properties
Content	The actual text, image or child elements. Its size is set by width and height.	width, height, min-width, max-width, min-height, max-height, overflow
Padding	Transparent space inside the border, between content and border. Takes the background of the element.	padding, padding-top, padding-right, padding-bottom, padding-left
Border	A line that goes around the padding and content.	border, border-width, border-style, border-color, border-radius, border-top ...
Margin	Transparent space outside the border that separates the element from its neighbours. Has no background.	margin, margin-top, margin-right, margin-bottom, margin-left, margin: auto

4. Importance of the box model

- **Layout design:** every layout (header, sidebar, cards) is made of boxes; knowing the model makes it possible to place them precisely.
- **Correct size calculation:** the total space taken by an element = content + padding + border + margin. A wrong assumption breaks layouts (e.g. two 50% columns with padding overflowing the row).
- **Spacing control:** padding controls the inner spacing and margin the outer spacing.
- **Visual styling:** borders, rounded corners, shadows and backgrounds are applied on the box.
- **Centring and alignment:** *margin: 0 auto* centres a block box horizontally.
- **Debugging and responsive design:** browser developer tools show the box model of every element; *box-sizing* makes fluid layouts easier.

5. Calculating the size of a box

For the default **box-sizing: content-box**:

- Total width = width + padding-left + padding-right + border-left + border-right + margin-left + margin-right
- Total height = height + padding-top + padding-bottom + border-top + border-bottom + margin-top + margin-bottom

Example: width = 300px; padding = 20px; border = 10px; margin = 25px.

Quantity	Calculation	Result
Visible box width (content + padding + border)	$300 + (20 \times 2) + (10 \times 2)$	360 px
Total space taken horizontally (with margin)	$360 + (25 \times 2)$	410 px

6. box-sizing property

Value	Meaning	Width of 300px results in
content-box	Default. width and height apply to the content only ; padding and border are added outside.	Visible width = 300 + padding + border.
border-box	width and height include the content, padding and border .	Visible width is exactly 300 px; the content shrinks.

7. Program to demonstrate all box model properties on a <div>

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>CSS Box Model</title>
<style>
  body { font-family: Arial, sans-serif; background: #eee; }

  #box1 {
    /* ----- CONTENT ----- */
    width: 300px;           /* content width */
    height: 120px;          /* content height */
    min-height: 100px;      /* smallest allowed height */
    max-width: 90%;         /* largest allowed width */
    overflow: auto;         /* scroll bar if content is larger */

    /* ----- PADDING (inside the border) ----- */
    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 10px;
    padding-left: 20px;     /* shorthand: padding: 10px 20px; */

    /* ----- BORDER ----- */
    border-width: 8px;
    border-style: solid;     /* dotted | dashed | double | groove ... */
    border-color: navy;
    border-radius: 15px;     /* rounded corners */
    border-top: 8px dashed red; /* one side different from the others */

    /* ----- MARGIN (outside the border) ----- */
    margin-top: 30px;
    margin-right: auto;
    margin-bottom: 30px;
    margin-left: auto;      /* auto left+right = horizontally centred */

    /* ----- OTHER RELATED PROPERTIES ----- */
    box-sizing: content-box; /* try border-box to see the difference */
    outline: 3px dotted green; /* drawn outside the border, no space */
    outline-offset: 5px;
    box-shadow: 6px 6px 10px gray;
    background-color: lightyellow; /* fills content + padding + border */
    background-clip: content-box; /* limit the background to the content */
  }
</style>
</head>
<body>
  <h2>CSS Box Model Demo</h2>

  <div id="box1">
    This is a div showing the content area, padding, border and margin.
    Use the browser's developer tools (F12) and inspect this element to see
    the box-model diagram with its exact sizes.
  </div>
</body>
</html>
```

8. Explanation

- **width, height** set the content box. *min-/max-* values limit how small or large it can become and *overflow* handles content that does not fit.
- **padding-*** creates space between content and border (10px top and bottom, 20px left and right).
- **border-width, border-style, border-color, border-radius** draw the border and round its corners. *border-top* overrides one side.
- **margin-*** creates the outer space; *auto* on the left and right margins centres the block within the page (works because a width is set).
- **Vertical margin collapsing:** when the bottom margin of one block touches the top margin of the next, only the *larger* of the two is used, not their sum. Horizontal margins never collapse.
- **outline** looks like a border but takes no space and does not affect layout.

- **Total width** of this div = 300 + 20 + 20 + 8 + 8 = 356px (margins auto).

9. Padding vs margin (viva point)

Basis	Padding	Margin
Position	Inside the border.	Outside the border.
Background	Shows the element's background.	Always transparent.
Negative values	Not allowed.	Allowed (pulls elements closer/overlaps).
Auto value	Not allowed.	Allowed (used for centring).
Collapsing	Never collapses.	Vertical margins of adjacent blocks collapse.
Purpose	Space between content and its border.	Space between the element and other elements.

10. Shorthand notation

Shorthand	Meaning
margin: 10px;	All four sides 10px.
margin: 10px 20px;	top-bottom 10px, left-right 20px.
margin: 10px 20px 30px;	top 10px, left-right 20px, bottom 30px.
margin: 10px 20px 30px 40px;	top, right, bottom, left (clockwise).
border: 2px solid red;	width, style and colour in a single line.

Q.5 (a)

Question: Create an employee login form as mentioned below: (i) Name (ii) Password (iii) Email (iv) Login and link this form to the form "employee information": (i) Phone number (ii) Gender (iii) Date of birth (iv) Address (v) Country (India, Outside India) using datalist (vi) Language known (Hindi, English) (vii) Department (Computer science, Mechanical, Electronics, MCA) using select list (viii) Upload documents (ix) Submit (x) Reset. Name and password should be mandatory fields, and limit the length of the phone number 10 digits.

10 Marks | CO1

Answer:

1. HTML forms

An **HTML form** collects input from the user and sends it to a server for processing. It is created with the `<form>` element that contains controls such as text boxes, radio buttons, check boxes, drop-down lists, file selectors and buttons.

Attributes of the `<form>` tag

Attribute	Description
action	URL of the page/program that receives the data (here: the employee information page).
method	<i>get</i> (data appended to the URL, visible, limited size) or <i>post</i> (data sent in the request body, used for passwords and files).
enctype	Encoding of the data. <i>multipart/form-data</i> is needed for file uploads.
name, id	Name / identity of the form for scripts and CSS.
target	Where to show the response (<code>_self</code> , <code>_blank</code> ...).
autocomplete	on / off - browser auto-fill.
novalidate	Disables the browser's built-in validation.

2. Form controls and attributes used in this program

Element / type	Purpose	Important attributes
<code>input type="text"</code>	Single-line text (name).	name, id, placeholder, required, maxlength, size, value
<code>input type="password"</code>	Masked text.	required, minlength, maxlength
<code>input type="email"</code>	Email with automatic format check.	required, multiple, placeholder
<code>input type="tel"</code>	Telephone number.	pattern, maxlength, minlength
<code>input type="radio"</code>	Choose one option of a group (gender).	same <i>name</i> , value, checked
<code>input type="checkbox"</code>	Choose many options (languages).	name, value, checked
<code>input type="date"</code>	Date picker (date of birth).	min, max, required
<code>textarea</code>	Multi-line text (address).	rows, cols, maxlength, placeholder
<code>input list + datalist</code>	Text box with suggested options - the user can also type a value.	list (id of datalist), option value
<code>select + option</code>	Drop-down list (department).	name, multiple, size, selected, value
<code>input type="file"</code>	File selector (upload documents).	accept, multiple, required
<code>input type="submit"</code>	Sends the form.	value
<code>input type="reset"</code>	Clears the form to its initial values.	value
<code>label</code>	Caption for a control; clicking it focuses the control.	for (id of the control)
<code>fieldset / legend</code>	Groups related controls with a title.	-

3. Program - File 1: login.html (employee login form)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Employee Login</title>
</head>
<body>
  <h2>Employee Login</h2>

  <!-- the form is linked to "employee_information.html" through the action attribute -->
  <form action="employee_information.html" method="get">

    <p>
      <label for="name">Name:</label>
      <input type="text" id="name" name="name" placeholder="Enter your name" required>
    </p>

    <p>
      <label for="pwd">Password:</label>
      <input type="password" id="pwd" name="password" minlength="6" required>
    </p>

    <p>
      <label for="email">Email:</label>
      <input type="email" id="email" name="email" placeholder="abc@example.com">
    </p>

    <p>
      <input type="submit" value="Login">
    </p>
  </form>
</body>
</html>
```

4. Program - File 2: employee_information.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Employee Information</title>
</head>
<body>
  <h2>Employee Information</h2>

  <form action="save.php" method="post" enctype="multipart/form-data">

    <!-- (i) Phone number : only 10 digits -->
    <p>
      <label for="phone">Phone Number:</label>
      <input type="tel" id="phone" name="phone" maxlength="10" minlength="10"
        pattern="[0-9]{10}" title="Enter exactly 10 digits" placeholder="10 digit number">
    </p>

    <!-- (ii) Gender : radio buttons -->
    <p>Gender:
      <input type="radio" id="m" name="gender" value="Male"> <label for="m">Male</label>
      <input type="radio" id="f" name="gender" value="Female"> <label for="f">Female</label>
      <input type="radio" id="o" name="gender" value="Other"> <label for="o">Other</label>
    </p>

    <!-- (iii) Date of birth -->
    <p>
      <label for="dob">Date of Birth:</label>
      <input type="date" id="dob" name="dob">
    </p>

    <!-- (iv) Address -->
    <p>
      <label for="addr">Address:</label><br>
      <textarea id="addr" name="address" rows="4" cols="40"></textarea>
    </p>

    <!-- (v) Country : datalist -->
    <p>
      <label for="country">Country:</label>
      <input list="countries" id="country" name="country" placeholder="Select or type">
      <datalist id="countries">
        <option value="India">
        <option value="Outside India">
      </datalist>
    </p>

    <!-- (vi) Language known : check boxes -->
    <p>Languages Known:
      <input type="checkbox" id="hi" name="lang" value="Hindi"> <label for="hi">Hindi</label>
      <input type="checkbox" id="en" name="lang" value="English"> <label for="en">English</label>
    </p>

    <!-- (vii) Department : select list -->
    <p>
      <label for="dept">Department:</label>
      <select id="dept" name="department">
        <option value="">-- Select --</option>
        <option value="cs">Computer Science</option>
        <option value="me">Mechanical</option>
        <option value="ec">Electronics</option>
        <option value="mca">MCA</option>
      </select>
    </p>

    <!-- (viii) Upload documents -->
    <p>
      <label for="docs">Upload Documents:</label>
      <input type="file" id="docs" name="documents" multiple accept=".pdf,.doc,.docx,.jpg,.png">
    </p>

    <!-- (ix) Submit and (x) Reset -->
    <p>
      <input type="submit" value="Submit">
```

```

        <input type="reset" value="Reset">
    </p>
</form>
</body>
</html>

```

5. Explanation

- **Linking the forms:** the login form has `action="employee_information.html"`, so when the *Login* button is pressed the browser opens the employee information page. (As an alternative, a hyperlink `<a href="employee_information.html">` can be used.)
- **Mandatory fields:** the *required* attribute on Name and Password stops the form from being submitted while they are empty and shows a browser message.
- **Phone number limited to 10 digits:** `maxlength="10"` does not allow more than 10 characters, while `pattern="[0-9]{10}"` allows exactly 10 digits and no letters.
- **Gender** uses radio buttons having the same *name*, so only one can be selected; **languages** use check boxes so both can be selected.
- **Country** uses *datalist*: the *list* attribute of the input refers to the *id* of the datalist, which supplies the suggestions India and Outside India.
- **Department** uses the *select* element with four *option* items.
- **File upload** needs `method="post"` and `enctype="multipart/form-data"`; *multiple* allows more than one file.
- **Submit** sends the data to the address in *action*; **Reset** clears all fields.

6. select vs datalist

Basis	select	datalist
Choice	User must choose only from the given options.	User can choose a suggestion or type any other value .
Tag structure	<select> containing <option> elements.	<input list="id"> plus a separate <datalist id="id">.
Multiple choice	Yes, with the <i>multiple</i> attribute.	No (single value).
Best for	Fixed set of values (department).	Suggestions (country, city).

Q.5 (b)

Question: Write CSS code that define two Id's and three classes of paragraph with different background, font color, margins, padding and border style.

10 Marks | CO2

Answer:

1. Selectors: id and class

CSS selectors choose the HTML elements to be styled. Two very common selectors are the **id selector**, written with a **hash (#)**, and the **class selector**, written with a **dot (.)**.

Basis	ID selector	Class selector
Symbol	# (hash)	. (dot)
HTML attribute	id="name"	class="name"
Uniqueness	Must be unique - used by only one element in a page.	Can be used by many elements.
Multiple names	One id per element.	An element can have several classes: <code>class="a b c"</code> .
Specificity	Higher (100).	Lower (10).
Example	<code>#intro { color: red; }</code>	<code>.note { color: blue; }</code>
JavaScript	<code>getElementById()</code>	<code>getElementsByClassName()</code>

2. Other property groups used

Property	Meaning and values
background / background-color	Colour or image behind the element.
color	Font (text) colour.
margin	Space outside the border; shorthand for margin-top/right/bottom/left.
padding	Space inside the border around the content.
border	Shorthand for <code>border-width border-style border-color</code> .
border-style	<i>solid, dashed, dotted, double, groove, ridge, inset, outset, none, hidden.</i>

3. Program

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>CSS id and class selectors on paragraphs</title>
<style>
/* ===== TWO ID SELECTORS (unique paragraphs) ===== */
#para1 {
  background-color: lightyellow;
  color: darkred;
  margin: 20px 40px;           /* top-bottom 20px, left-right 40px */
  padding: 15px;
  border: 3px solid red;
}
#para2 {
  background-color: #e0f7fa;
  color: #004d40;
  margin-top: 10px;
  margin-left: 80px;
  padding: 10px 25px;
  border: 4px double teal;
}

/* ===== THREE CLASS SELECTORS (reusable) ===== */
p.dashed {
  background-color: #fce4ec;
  color: #880e4f;
  margin: 15px 0 15px 30px;
  padding: 12px;
  border: 2px dashed #c2185b;
}
p.dotted {
  background-color: #e8f5e9;
  color: #1b5e20;
  margin: 25px;
  padding: 5px 20px 5px 20px;
  border: 3px dotted green;
  border-radius: 10px;
}
p.grooved {
  background-color: #ede7f6;
  color: #311b92;
  margin: 10px 60px 30px 10px;
  padding: 20px;
  border: 6px groove #7e57c2;
  font-style: italic;
}
</style>
</head>
<body>

<h2>Two ids and three classes</h2>

<p id="para1">This is paragraph 1 styled using the id selector #para1.</p>
<p id="para2">This is paragraph 2 styled using the id selector #para2.</p>

<p class="dashed">Paragraph with class "dashed".</p>
<p class="dotted">Paragraph with class "dotted".</p>
<p class="grooved">Paragraph with class "grooved".</p>

<!-- classes are reusable : same class on another paragraph -->
<p class="dashed">Another paragraph using the same "dashed" class.</p>

</body>
</html>
```

4. Explanation

Selector	Background	Font colour	Margin	Padding	Border style
#para1	light yellow	dark red	20px 40px	15px	3px solid red
#para2	light cyan	dark green	top 10px, left 80px	10px 25px	4px double teal
p.dashed	light pink	dark pink	15px 0 15px 30px	12px	2px dashed
p.dotted	light green	dark green	25px (all sides)	5px 20px	3px dotted, rounded
p.grooved	light purple	dark violet	10px 60px 30px 10px	20px	6px groove

- The **id selectors** (#para1, #para2) are used once each because an id must be unique in a page.
- The **class selectors** (p.dashed, p.dotted, p.grooved) are written as *p.classname* so they apply only to paragraphs; they can be used on any number of paragraphs (the dashed class is used twice).
- Each rule uses a **different** background colour, font colour, margin, padding and border style, as required by the question.
- If an element has both an id and a class, the **id rule wins** when both set the same property, because it has higher specificity.

Bonus Revision Notes - Scripting Languages (Full Syllabus Quick Review)

The topics below are related to this paper and are commonly asked in Scripting Languages examinations (HTML, CSS and a first look at JavaScript).

A. Scripting language vs programming language

Basis	Scripting language	Programming language
Execution	Interpreted line by line at run time (by browser or server).	Usually compiled to machine code first.
Purpose	Automate tasks, add behaviour to existing applications, web pages.	Build complete standalone software.
Speed	Slower in execution, quick to write and test.	Faster execution.
Examples	JavaScript, PHP, Python, Perl, Ruby, Bash, VBScript.	C, C++, Java, C#.
Client / server	Client-side: JavaScript. Server-side: PHP, Python, Node.js.	-

B. HTML vs CSS vs JavaScript

Technology	Role	Analogy
HTML	Structure and content of the page.	Skeleton of the body.
CSS	Presentation - colours, fonts, layout.	Skin and clothes.
JavaScript	Behaviour and interactivity - events, validation, DOM changes.	Muscles and brain.

C. HTML entities (special characters)

Character	Entity	Character	Entity
< (less than)	<	> (greater than)	>
& (ampersand)	&	" (double quote)	"
Copyright	©	Non-breaking space	
Registered	®	Trademark	™

D. Hyperlinks and images

```
<a href="page.html" target="_blank" title="Open page">Text link</a>
<a href="#section2">Jump inside the same page</a>          <!-- bookmark; <h2 id="section2"> -->
<a href="mailto:abc@example.com">Send mail</a>
<a href="tel:+911234567890">Call</a>

<a href="big.html"></a> <!-- image as a link -->
```

E. Basic form input types (HTML5)

Type	Use	Type	Use
text	Single line text	number	Numeric input with min/max/step
password	Hidden characters	range	Slider
email	Email address check	color	Colour picker
url	Web address	date, time, datetime-local	Date and time pickers
tel	Phone number	month, week	Month / week pickers
search	Search box	hidden	Invisible data field
radio / checkbox	One / many choices	file	Upload
submit / reset / button	Buttons	image	Image as submit button

F. CSS selectors summary

Selector	Example	Selects
Universal	* { }	All elements.
Element	p { }	All p elements.
Class	.note { }	Elements with class="note".
ID	#top { }	The element with id="top".
Group	h1, h2, p { }	All listed elements.
Descendant	div p { }	All p inside a div (any depth).
Child	div > p { }	p that are direct children of div.
Adjacent sibling	h1 + p { }	The p immediately after h1.
General sibling	h1 ~ p { }	All p siblings after h1.
Attribute	input[type="text"] { }	Inputs whose type is text.
Pseudo-class / element	a:hover, p::first-line	State / part of an element.

G. CSS units and colour formats

Absolute units	Relative units	Colour formats
px, cm, mm, in, pt, pc	%, em, rem, vw, vh, ch	name (red), hex (#ff0000), rgb(255,0,0), rgba(255,0,0,0.5), hsl(0,100%,50%)

H. JavaScript primer

JavaScript is a client-side scripting language that runs in the browser. It can be written inside `<script>` tags or in an external `.js` file.

```
<script>
var a = 10; let b = 20; const PI = 3.14;    // variables
document.write("Sum = " + (a + b));        // output on the page
console.log("Hello");                      // output in the browser console
```

```

alert("Welcome");                                // pop-up message

function square(n) { return n * n; }              // function
if (a > 5) { alert("greater"); } else { alert("smaller"); }
for (var i = 1; i <= 5; i++) { document.write(i + " "); } // loop

// DOM + event + form validation
function validate() {
    var name = document.getElementById("name").value;
    if (name == "") { alert("Name must be filled out"); return false; }
    return true;
}
</script>
<form onsubmit="return validate()">
    <input type="text" id="name"> <input type="submit">
</form>
<button onclick="document.getElementById('demo').innerHTML = 'Clicked!'">Click</button>
<p id="demo"></p>

```

I. Most probable exam questions (practice list)

- Explain the different types of lists in HTML with attributes and nested lists.
- Create a registration form with validation using HTML5 attributes (required, pattern, min, max).
- Explain HTML tables with colspan and rowspan; create a time-table.
- Explain hyperlinks (internal, external, bookmark, mailto) and image links.
- Explain CSS selectors and CSS specificity with examples.
- Explain CSS text, font, background, list and table properties.
- Explain CSS float, display and overflow properties.
- Create a navigation bar (horizontal menu) using an unordered list and CSS.
- Explain video and audio embedding in HTML5; compare <iframe>, <embed> and <object>.
- Write a JavaScript program for form validation.